Laboratorio 05
Pruebas unitarias y dobles de prueba

Ingeniería de Software II · eShopOnWeb · 26 de octubre de 2026

1 Objetivo y proyecto utilizado

Se implementaron pruebas para el carrito de eShopOnWeb en C# y ASP.NET Core. Se verificaron el dominio, las consultas del repositorio y los servicios de aplicación. Las **42 pruebas nuevas aprobaron** y se corrigió un desbordamiento de cantidad mediante TDD.

La base es el proyecto público de NimblePros, conservando su licencia MIT. Se utilizó el commit `f21d623`; el aporte propio consiste en las pruebas, la corrección, los reportes y las evidencias.

2 Arquitectura y entorno

DDD: Basket es la raíz del agregado y encapsula BasketItem. **Arquitectura limpia:** ApplicationCore contiene las reglas e interfaces; Infrastructure implementa EfRepository; Web presenta la tienda. BasketService depende de IRepository<Basket>, que puede sustituirse por un doble.

Herramientas: SDK .NET 10.0.403, xUnit 3.2.2, NSubstitute 6.0.0, Coverlet 10.0.1 y ReportGenerator 5.5.11. Las pruebas están separadas en tests/Lab05.Tests. Para integración se utilizó SQLite temporal.

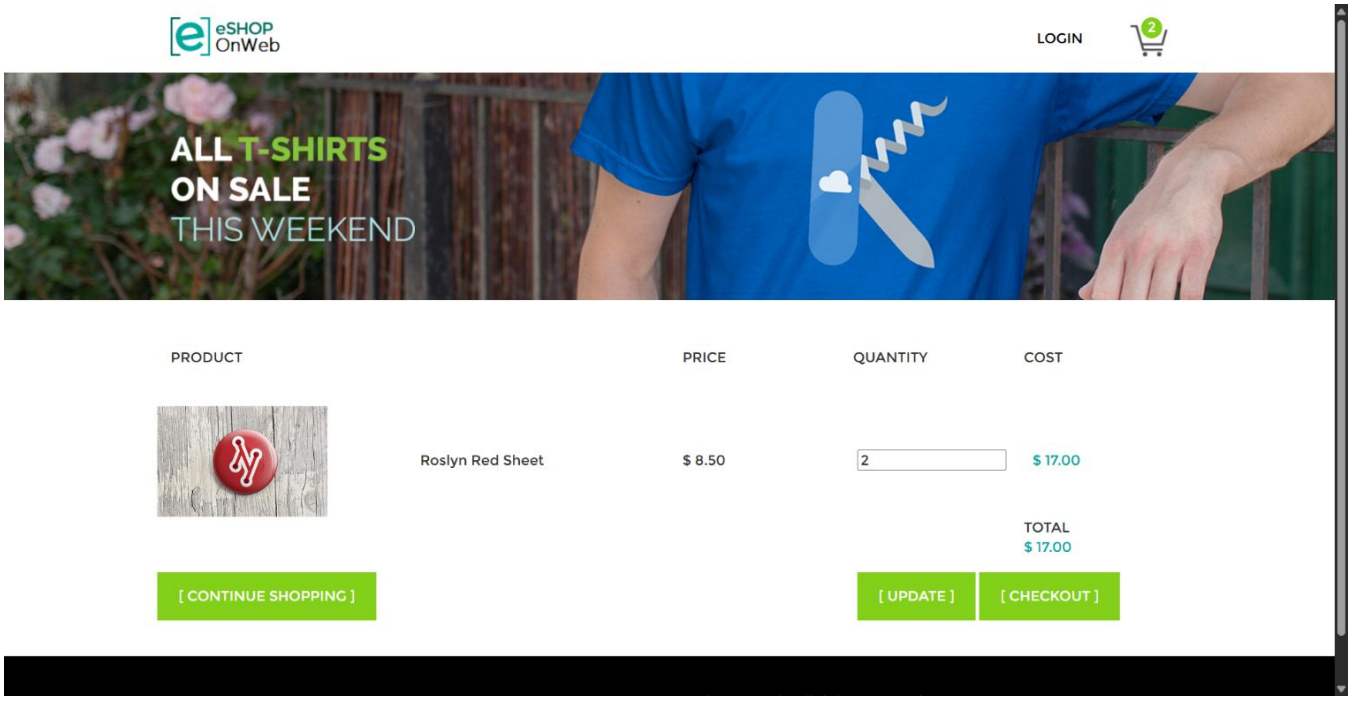


Figura 1. Carrito en ejecución local: 2 unidades × $8.50 = $17.00. Se comprobó la actualización desde el navegador.

3 Casos de prueba y preparación

Se definieron 42 ejecuciones: 19 de dominio, 4 de especificaciones del repositorio, 12 de servicios con dobles y 7 de integración. La tabla resume casos representativos; la especificación completa está en docs/lab05/casos-de-prueba.md.

Clase o método	Caso y valores	Resultado esperado
Basket.AddItem	Mismo producto; cantidades 2 y 3	Un único ítem con cantidad 5
Basket.RemoveEmptyItems	Ítems con cantidades 0 y 2	Se elimina el de cantidad 0
BasketItem.SetQuantity	Cantidad nueva -1	Excepción; estado sin cambios
BasketItem.AddQuantity	2147483647 + 1	Excepción; evita desbordamiento
BasketWithItemsSpecification	ID 7 frente a entidades 7 y 8	Coincide solo el ID solicitado
EfRepository.Add / Update	Guardar 2; actualizar a 5; releer	Datos persistidos entre contextos
EfRepository.Delete	Carrito con un ítem	Elimina carrito e ítem dependiente
BasketService. AddItemToBasket	Repositorio devuelve null	Crea y actualiza una sola vez
BasketService. SetQuantities	ID inexistente o cantidad -1	NotFound o excepción; no guarda
BasketService. TransferBasketAsync	Anónimo con 2; comprador con 3	Fusiona a 5 y elimina anónimo

Inicializar, ejecutar y verificar: se usó Arrange-Act-Assert, Fact, Theory y aserciones. El constructor crea una instancia nueva por prueba. En integración, Dispose cierra el contexto y la conexión al finalizar cada caso.

Dobles: NSubstitute simula IRepository<Basket> y el registrador. Returns controla respuestas; Received(1) y DidNotReceive verifican interacciones. Las 7 pruebas con SQLite ejercitan el repositorio real y dos flujos completos de BasketService.

4 Ciclo TDD y resultados de ejecución

RED: con cantidad máxima, AddQuantity(1) no lanzaba la excepción esperada. **GREEN:** se comprobó la capacidad antes de sumar. **REFACTOR:** la validación se unificó en Guard.Against.OutOfRange con límite int.MaxValue - Quantity.



Laboratorio 05 · Ciclo TDD real

Invariante: una suma no debe desbordar la cantidad ni dejarla negativa.

RED · implementación original · 1 prueba fallida

```
[xUnit.net 00:00:02.34]     Lab05.Tests.Domain.BasketItemTests.AddQuantityRejectsOverflowAndPreservesQuantity [FAIL]
  Con error Lab05.Tests.Domain.BasketItemTests.AddQuantityRejectsOverflowAndPreservesQuantity [371 ms]
  Mensaje de error:
   Assert.Throws() Failure: No exception was thrown
Expected: typeof(System.ArgumentOutOfRangeException)
Archivo de resultados: <lab05>\evidence\red\red.trx

Con error! - Con error:     1, Superado:     0, Omitido:     0, Total:     1, Duración: 401 ms - Lab05.Tests.dll (net10.0)
```

GREEN · validación incorporada · 1 prueba aprobada

```
Serie de pruebas para <repo>\tests\Lab05.Tests\bin\Debug\net10.0\Lab05.Tests.dll (.NETCoreApp,Version=v10.0)
1 archivos de prueba en total coincidieron con el patrón especificado.
Archivo de resultados: <lab05>\evidence\green\green.trx

Correctas! - Con error:     0, Superado:     1, Omitido:     0, Total:     1, Duración: 34 ms - Lab05.Tests.dll (net10.0)
```

REFACTOR · validación unificada con Guard

```
public void AddQuantity(int quantity)
    {
        // Validate against the remaining capacity before modifying the entity.
        Guard.Against.OutOfRange(quantity, nameof(quantity), 0, int.MaxValue - Quantity);

        Quantity += quantity;
    }
```

Fragmentos de registros originales. Las rutas locales se abreviaron para facilitar la lectura. La suite completa posterior aprobó 42/42 casos.

Figura 2. Vista HTML de fragmentos de los registros reales: una prueba fallida, la misma prueba aprobada y la refactorización. Se abreviaron las rutas locales.



Laboratorio 05 · Resultados de ejecución

eShopOnWeb · módulo carrito · xUnit + NSubstitute + SQLite · 26 de octubre de 2026

42	42	0	0
pruebas ejecutadas	aprobadas	fallidas	omitidas

Distribución de pruebas

Grupo	Casos	Resultado
Application	12	12 aprobadas
Domain	19	19 aprobadas
Integration	7	7 aprobadas
Repository	4	4 aprobadas

Ejecución reproducible

```
dotnet test tests/Lab05.Tests/Lab05.Tests.csproj
 --settings lab05.runsettings --collect "XPlat Code Coverage"
 --logger "trx;LogFileName=lab05.trx"
```

Fuente: archivo lab05.trx generado por dotnet test. La vista organiza los resultados sin modificar sus valores.

Detalle de las 42 ejecuciones

Prueba	Resultado
Lab05.Tests.Application.BasketServiceTests.AddCreatesBasketWhenBuyerHasNone	Passed

Figura 3. Resumen leído del archivo TRX: 42 aprobadas, 0 fallidas y 0 omitidas.

Como regresión adicional se ejecutaron las **20 pruebas originales del carrito**, también aprobadas. Los archivos RED, GREEN y finales se conservan en docs/lab05/evidence.

5 Cobertura de código

Coverlet registró la ejecución y ReportGenerator generó los reportes HTML. En las cinco clases seleccionadas del carrito se alcanzaron **107/107 líneas y 18/18 ramas**. Este porcentaje corresponde al módulo evaluado.

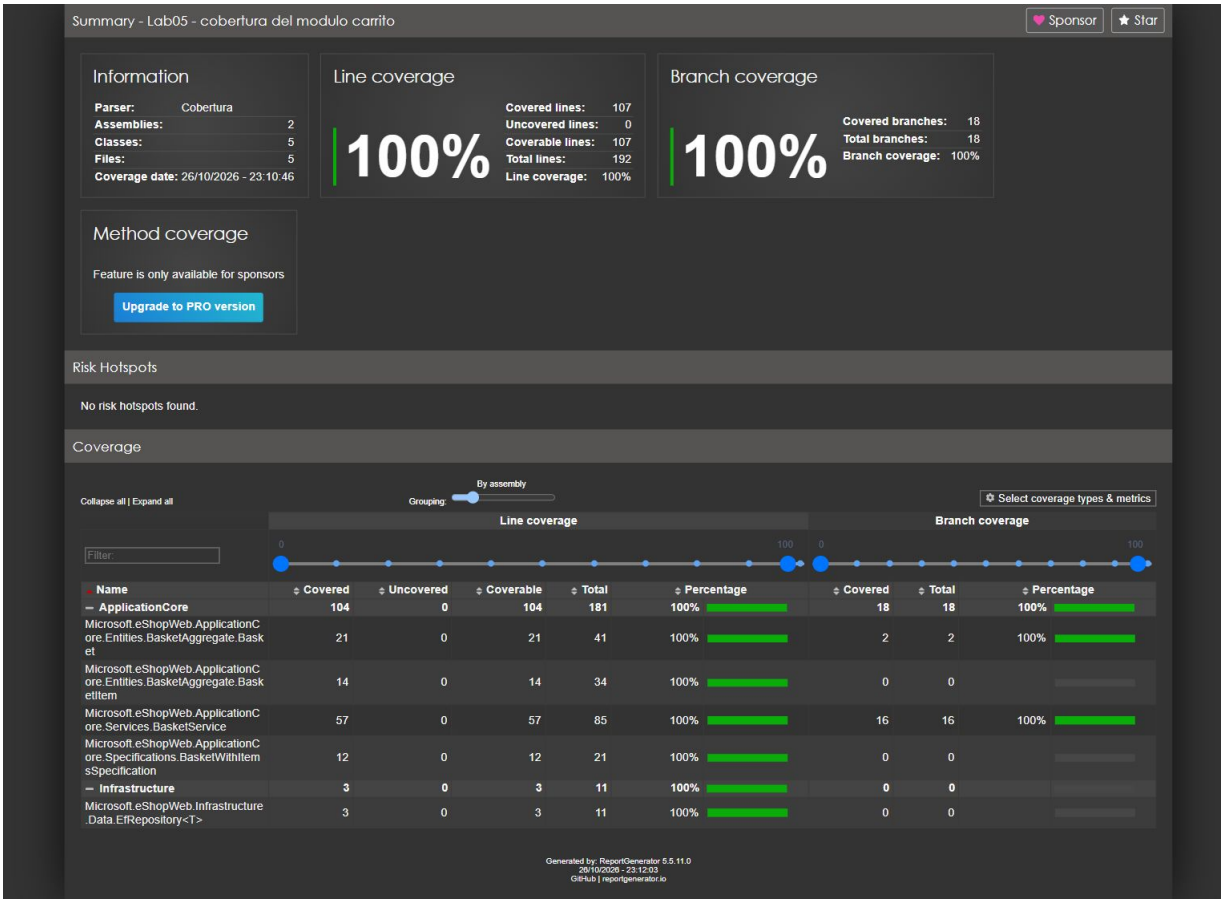


Figura 4. Reporte filtrado a Basket, BasketItem, BasketService, BasketWithItemsSpecification y EfRepository.

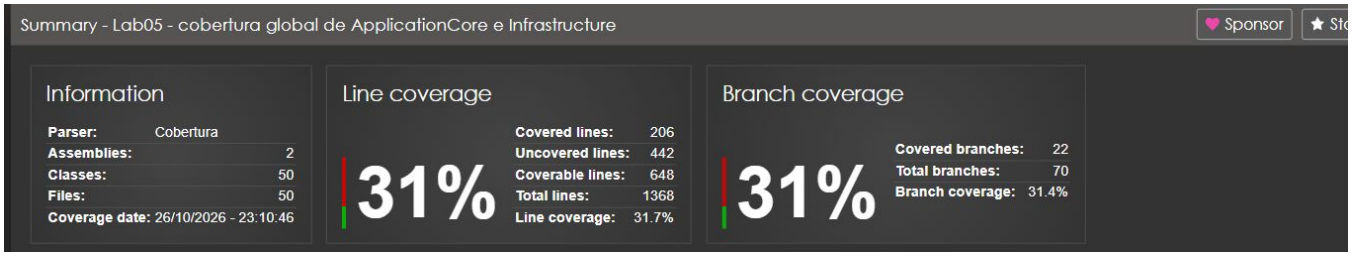


Figura 5. Conjunto medido ApplicationCore + Infrastructure: 206/648 líneas (31,7%) y 22/70 ramas (31,4%).

La medición global excluye migraciones y código generado; otros módulos no fueron el objetivo del laboratorio. El 100% de EfRepository corresponde a sus 3 líneas propias: las operaciones heredadas de Ardalis pertenecen a una biblioteca externa.

SQLite permite verificar persistencia relacional, pero no reemplaza pruebas específicas de SQL Server. Se adaptaron únicamente las secuencias HiLo de catálogo en el contexto de prueba. La restauración también informó avisos de vulnerabilidades en dependencias; no se realizó una actualización integral de seguridad.

6 Organización del trabajo y repositorio

Repositorio de entrega: https://github.com/GinoSebastianD/lab5-eshoponweb. Se publicaron las ramas main y development, conservando la rama de trabajo feature/lab05-basket y los commits del ciclo TDD.

Tablero privado de la cuenta: https://github.com/users/GinoSebastianD/projects/1. Estados: TO-DO, CURRENT ITERATION, IN PROGRESS, FIX VALIDATION y DONE. Los cuatro issues del repositorio son públicos.

Tarea	Etiqueta	Estimación
#1 Dominio y corrección del desbordamiento	enhancement	60 min
#2 Especificaciones y persistencia	unit test	60 min
#3 Servicios con dobles e integración	unit test	70 min
#4 Ejecución, cobertura e informe	unit test	50 min

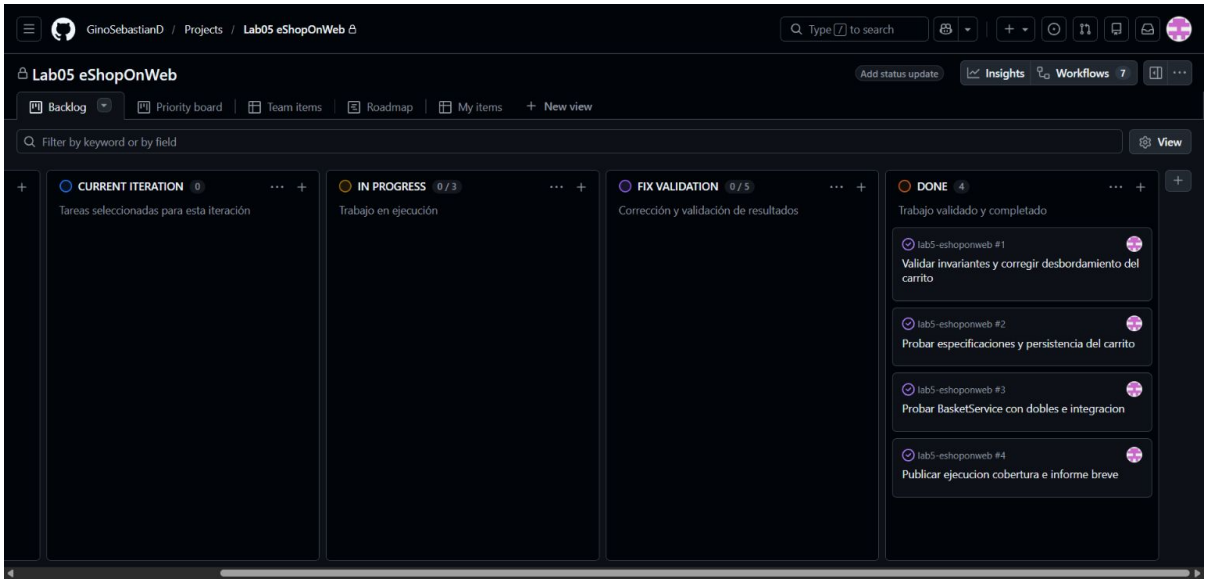


Figura 6. Tablero GitHub: los cuatro issues cerrados aparecen en DONE. Publicación realizada el 27 de octubre de 2026.

7 Conclusión y entregables

El carrito cumple los casos evaluados y conserva sus pruebas anteriores. Los dobles permitieron aislar los servicios, mientras SQLite comprobó la persistencia. Se entregan el código, la especificación de casos, los registros TRX, la cobertura HTML y este informe con capturas.

Referencias

Guía del curso: Lab05 Unit Testing Test Doubles, actividades 1 a 7.
Proyecto base: github.com/NimblePros/eShopOnWeb.
Arquitectura: documentación oficial de eShopOnWeb.